

First subproblem

The first subproblem was just a reward for chewing through the problem statement. We can copy all the digits from the statement, generate all the rotations & their flipped versions, and compare them to each input. One of them will match exactly, and we can answer with that.

Second subproblem

The second subproblem consists of only 10 test cases, and each image fits comfortably without line-wrapping in your favourite text editor.

We can therefore solve this subproblem by hand, or perhaps more accurately, by eye. (Zooming out might help!)

The general approach and even digits

When it comes to problems where some structure is being modified, it is often useful to find some **invariants** - properties of the structure that do not change as a result of the modifications.

This problem is all about finding what invariants our digits have with respect to the three operations, so that we can then measure and distinguish the digits based on them.

One easy to spot invariant with respect to the three operations in this task is the number of four-connected components.

Consider some square in the image with character c (either $.$ or $\#$). We will call its four-connected component (or just 'component') the maximal set of characters c , that can be reached from it by making moves in the four cardinal directions, never stepping on a different character than c .

It is trivial to see that rotations and flips do not change the number of components in the image.

Can a distortion do so? It cannot decrease the number of components, as it only adds characters. On the other hand, each inserted character must be the same as the one above or below it, so for example whenever we insert a $\#$, it has to be adjacent to an original $\#$ and thus it will belong to the same component as that original $\#$. (And if it has a $\#$ both above and below, those already were in the same component and now they have to remain in the same component as we have to insert $\#$ between them.)

So, for example two will have five components: three components of $\#$ (the letters), and two components of $.$ (the surrounding background, and the gap in the o).

Here is a table of the number of components for each digit, per character, ordered by the number of $\#$ components, then $.$ components.

Digit	$\#$	$.$
two	3	2
one	3	3
six	4	1
four	4	2

Digit	#	.
zero	4	3
five	5	2
nine	5	2
three	5	3
seven	5	3
eight	6	3

We can see that just by counting the number of # and . components, we can distinguish all digits except five from nine, and three from seven.

Luckily, these are all odd, so this is enough for the third subproblem. Use a graph algorithm of your choice, e.g. BFS or DFS, to count the number of components of each type, and use it to distinguish the even digits.

The final subproblem

We have to utilize a few more invariants on top of the number of components to distinguish the final digits from each other. There are a variety of more subtle invariants; we will describe one valid set that is sufficient.

Holes

The first thing to note is that each . component, other than the background, is a hole in some component of #, and each such # component has at most 1 such hole. We will check for this and remember for each # component whether it has a hole.

Overlaps

The other property we will utilize is whether two # components overlap; you can think of this as whether one component would cast a shadow on the other, if we shone light on the image from some direction (either horizontally, or vertically). To check whether two components overlap vertically, find the first and last column of each component, and check whether these intervals are disjoint (no overlap) or not (overlap). Analogously we can check for a horizontal overlap by looking at their lowest and highest rows.

Let's check this property against our operations.

Flips do not change overlaps.

Distortions also cannot change overlaps. They only add characters, so yet again they cannot remove an overlap. When it comes to adding an overlap, we would need to have two components that almost overlap (one ends where the other begins), so that after a distortion they would overlap. But in our starting images, every non-overlapping component is separated by a column of . vertically, and horizontally the dot on the i is separated by a . from its leg. This separation will always remain, so no overlaps can be introduced.

Overlaps do actually change when we perform a rotation. However, they will change in a particular way - two components will overlap horizontally after a rotation if and only if they overlapped vertically before, and vice-versa. The number of overlaps remains constant, just the sets of vertical and horizontal overlaps swap.

We will use this to our advantage. Notice that vertically, the only components that overlap are the dot on the `i` and its leg. Conversely, there are always multiple overlaps between the letters in the horizontal direction. So, count the number of vertical overlaps between `#` components. If we have one or fewer, we know the image is oriented horizontally (though it might be flipped). If we have more than one, it is oriented vertically (it ended up rotated).

Now that we know in which direction the image is oriented, we can order our components in what used to be the horizontal direction. We won't know if this ordering is left-to-right or right-to-left, but we don't need to know this.

Time to distinguish

Now that we are armed with some additional properties, we are able to distinguish the final two pairs of digits.

The distinguishing characteristic between three and seven is which components have the holes. When ordered horizontally, in seven the holes are always in the component second from the left, and second from the right. In three, the holes are in one component on the edge, and its neighbor.

We can check which one is the case, and use that to distinguish the three from seven.

The distinguishing characteristic between five and nine is that the dot on the `i` overlaps horizontally with one of the letters in five (the `f`), but none of the letters in nine.

We can find which component is first letter (the `f` or `n`) – the component on the opposite end should be an `e`, which has a hole.

The dot and leg of the `i` will be the next two neighboring components. We can check whether they both overlap with our 'first' letter. If they do, it's an `f`, so the digit is a five. Otherwise it's a nine.

Complexity

Finding all the components, including keeping track of their lowest/highest coordinates, and checking whether they have a hole, is $O(rc)$.

Checking overlaps and ordering the components scales with the number of components, which in this task is a small constant.

So our total complexity is $O(rc)$.